

Ada Crib Sheet

OBJEKTUM LIMITED

Airport House, Purley Way, Croydon, Surrey, CR0 0XZ, United Kingdom

t: +44 (0)845 199 9932 e: info@objektum.com w: www.objektum.com

CONTENTS

Introduction 2
Intended Audience 2
Predefined Types 3
Simple Numeric Types 3
Enumeration Types 3
Real Types 3
Simple Array Types 4
Unconstrained Array Types 4
Record Types 4
Variant Record 5
Tagged Record Types 5
Discriminated Record Types 5
Subprograms & Operators 6
Access Types 6
Program Control Flow 7
Package Specification 8
Package Body 8
Hierarchical Libraries 9
Generic Specifications 9
Generic Instantiations 9
Tasking 10
Exceptions 11
The Predefined Exceptions 11
Declaring and Handling Exceptions 11
Representation 11
Attributes 12
Discrete Types 12
Array Types 12

Introduction

This document provides an overview of the common parts of the Ada95 language. The language is vast and therefore this document does not cover the entire language, but the syntax most commonly used by developers.

This document has been structured in a similar manner to the Objektum Ada course notes and therefore provides a useful Ada syntax reference manual.

The text colours have been chosen to match the defaults used with the Ada GIDE development environment. All Ada reserved words are in lower case and are shown in blue. All Identifiers start with an upper case letter and are shown in black. All comments are shown in Green.

Intended Audience

This document is aimed at software engineers who are becoming familiar with the Ada programming language. It does not provide programming language examples, but should be used as a quick reference guide when attempting to remember the Ada syntax.

Predefined Types

Integer	-	Permit values that are whole numbers
Float	-	Permit real number values
Duration	-	Fixed point type, represents periods in seconds
Character	-	Enumeration type represents the character set
String	-	Represents collections of characters
Boolean	-	Represents Boolean values (True / False)

Simple Numeric Types

```
type My_Signed_Type is range –2**15 .. 2**15-1 ;

type My_Unsigned_Type is range 0 .. 1000 ;

subtype My_Subtype is My_Unsigned_Type range 1 .. My_Unsigned_Type'Last ;

My_Object_1 : My_Signed_Type ;                                  -- Uninitialised variable
My_Object_2 : My_Signed_Type := 0 ;                             -- Variable, initial value of 0
My_Object_3 : constant My_Signed_Type := My_Signed_Type'First ;  -- Constant value of –2**15
```

Enumeration Types

```
type Day_Type is (Mon , Tue , Wed , Thu , Fri , Sat , Sun);

subtype Weekday_Type is Day_Type range Mon .. Fri ;

Today : Day_Type := Tue ;                                  -- Variable, initial value = Tue
Tomorrow : constant Weekday_Type := Weekday_Type'Last ;    -- Constant, value = Fri
```

Real Types

```
type My_Float_Type is digits 7 ;

type My_Float_Type is digits 7 range 0.0 .. 10.0;

subtype My_Float_Subtype is My_Float_Type range 0.0 .. My_Float_Type'Last ;

type My_Fix_Type is delta 0.1 range –1.0 .. +1.0 ;

My_Float : Float_Type := 0.0 ;            -- Variable of My_Float_Type, Initial value = 0.0
My_Fix : constant My_Fix_Type := 0.5 ;    -- Constant My_Fix_Type, value = 1.5
```

Simple Array Types

```
type One_Dimensional_Array_Type is array (Index_Type) of Element_Type;

type Two_Dimensional_Array_Type is array (Index_1_Type , Index_1_Type) of Element_Type;


My_1_Dimensional_Array : One_Dimensional_Array_Type := (others => 0); -- Initialise all elements

My_2_Dimensional_Array : Two_Dimensional_Array_Type := (  1 => (1 => 01 , 2 => 03 , 4 => 07),
                                                          2 => (1 => 10 , 2 => 13 , 4 => 17),
                                                          3 => (1 => 20 , 2 => 23 , 4 => 27) );

My_Anonymous_Array : array (1..6) of Element_Type ;  -- Uninitialise anonymous array


My_Anonymous_Array (1)    := 0 ;                          -- Array element access
My_Anonymous_Array (1..2) := My_Anonymous_Array (5..6);   -- Array slice
```

Unconstrained Array Types

```
type Unconstrained_Array_Type is array (Index_Type range <>) of Element_Type ;

type Two_Dimensional_Unconstrained Array_Type is array
  (Index_1_Type range <>, Index_1_Type range <>) of Element_Type;

subtype Constrained_Subtype is Unconstrained_Array_Type (1..5);


My_Unconstrained_Array : Unconstrained_Array_Type (1..3) ;  -- Constrained to 3 elements
```

Record Types

```
type Date_Type is
record
    Day   : Day_Type   ;
    Month: Month_Type ;
    Year  : Year_Type  ;
end record;

The_Date : Date_Type ; -- Uninitialised variable
Christmas : constant Date_Type := (Day   => 25 ,
                                   Month => 12 ,
                                   Year  => 2002);  -- Constant initialised using named aggregate

The_Date.Month := 01 ; -- Record component access
```

Variant Record

```
type Car_Type ( Is_Sports : Boolean := False ) is   -- Optional Default
record
   Top_Speed : Natural;

   case Is_Sports is
      when True  => Roof_Down : Boolean ;
      when False => No_Of_Seats : Positive ;
   end  case;
end record;

Ferrari     : Car_Type (Is_Sports => True) ;    -- Constrained object (always a sports car)
Escort      : Car_Type (Is_Sports => False) ;   -- Constrained object (never a sports car)
Hot_Rod : Car_Type ;                            -- Can be either (Type has default applied)
```

Tagged Record Types

```
type Car_Type is tagged
record
   Top_Speed : Natural;
end record;

type Sports_Car is new Car_Type with
record
   Roof_Down : Boolean ;
end record ;
```

Discriminated Record Types

```
type Square_Type ( Size : Index_Type := 0 ) is  -- Optional Default
record
   Data : Two_Dimensional_Unconstrained Array_Type (1..Size, 1..Size);
end record ;

Fixed_Square : Square_Type (Size => 4);     -- Will always by 4 * 4
Mutable_Square : Square_Type ;              -- Can be any size within the bounds of Index_Type
```

Subprograms & Operators

```
function  My_Function (Param : in Param_Type) return Some_Type ;

procedure My_Procedure (  Param_1 : in        Param_Type  ;
                          Param_2 : out       Param_Type  ;
                          Param_3 : in out    Param_Type  ;
                          Param_4 : access    Param_Type  );

function "=" (L,R : Some_Type) return Boolean ;
```

Access Types

```
type Some_Ptr_Type is access Some_Type ;

type General_Access_Type is access all Some_Type ;

type Constant_Access_Type is access constant Some_Type ;

type Procedure_Acceess_Type is access procedure (Param : in out Param_Type) ;

type Function_Acceess_Type is access function (Param : in Param_Type) return Some_Type ;

My_Ptr : Some_Ptr_Type ;                          -- Object of the access type, value = null
My_Ptr : Some_Ptr_Type := new Some_Type;           -- Dynamically create object, pointed at
My_Ptr : Some_Ptr_Type := new Some_Type' (Value);  -- New object has an initial value

Object : aliased Some_Type ; -- Statically declared object, that can be indirectly referenced

My_Ptr : General_Access_Type := Object'Access ;   -- My_Ptr points to statically declared object
My_Ptr.all := New_Value ;                         -- Change the value of the object via My_Ptr
```

Program Control Flow

```
If X = 0.0
then
   -- Do Something
else
   -- Do something else
end if;
```

```
If X = 0.0
and then Y > 1.0
then
   -- Do Something
else
   -- Do something else
```

```
If X = 0.0
then
   -- Do Something
elsif Y > 1.0
then
   -- Do something else
end if;
```

```
case Today is
   when Mon..Wed  => Do_Mon_to_Wed_Stuff ;
   when Thur | Fri   => Do_Thurs_or_Fri_Stuff ;
   when Sat          => Do_Sat_Stuff ;
   when others       => Do_Stuff_For_Other_Days ;
end case ;
```

```
subtype Weekday is Day_Type range Mon..Fri ;
subtype Weekend is Day_Type range Sat..Sun ;

case Today is
   when Weekday   => …..
   when Weekend   => ……
end case ;
```

```
Optional_Label:
loop
   -- Statement(s)
end loop Optional_Label;
```

```
for X in 1 .. 100
loop
   -- Statement(s)
end loop;
```

```
for X in reverse Discrete_Type
loop
   -- Statement(s)
end loop;
```

```
while Today /= Sat
loop
   -- Statement(s)
end loop;
```

```
<<Devil>>

   -- Statement(s)

goto Devil;
```

```
This_Loop:
for X in 1 .. 100
loop
   -- Statement(s)
   exit This_Loop when Y = 100 ;

end loop;
```

Package Specification

```
with Other_Package;    -- Context Clause
package Package_Name is

   type Exported_Private_Type is private ;
   type Exported_Limited_Type is limited private;

   function    Get_Stuff (Param : in Param_Type) return Stuff_Type;
   procedure Do_Stuff (Param : out Param_Type);

   -- Other visible Declarations (constants, types and subprograms)

private

   type Exported_Private_Type is  .....   -- Full type definition
   type Exported_Limited_Type is  .....   --  “     “       “   “

end Package_Name;
```

Package Body

```
with Other_Package;
package body Package_Name is

   -- Package level declarations (constants, types, objects and subprograms)

   --
   -- Implementation of operations defined in the package specification
   --
   function Get_Stuff (Param : in Param_Type) return Stuff_Type is
   begin
      --  Implementation of function

      return Some_Value ;

   end Get_Stuff;

   procedure Do_Stuff (Param : out Param_Type) is separate ; -- Implemented in a separate file

begin
   -- Optional elaboration code

end Package_Name;
```

Separates

```
separate (Package_Name)
procedure Do_Stuff (Param : out Param_Type) is
begin
   --  Implementation of procedure
   Param := Some_Value ;
end Do_Stuff ;
```

Hierarchical Libraries

```
package Parent.Public_Child_Name is …..
private package Parent.Private_Child_Name is …..
```

Generic Specifications

```
generic

    type Item_Type is private ;  -- Formal Parameters (Can include objects and subprograms)

procedure Swap (X,Y : in out Item_Type) ;  -- Could be a function (body implementation as normal)



generic

   type Generic_Type is range <> ;  -- Any Integer type (could be one of many possible types)

   Object : in Some_Type ;  -- Mode could be 'in out'

   with procedure My_Procedure (Param : in Param_Type);

   with function "<" (L,R : in Generic_Type) return Boolean is <>;  -- Default operator

package Generic_Package is ….  -- Package Specification as normal
```

Generic Instantiations

```
procedure Swap_Integers is new Swap (Item_Type => Integer);  -- Generic Procedure Instantiation

package My_Instantiation is new Generic_Package
   (Generic_Type    => Some_Integer_Type,        -- Actual type must match expected type
    Object          => Some_Value,               -- Object must be of the correct type
    My_Procedure    => Some_Procedure);          -- Procedure signature must match
```

Tasking

```
task Task_Name is  -- could be a task type (by inserting 'type' before task name
   entry Entry_1 ;                                    -- Entry point with no parameters
   entry Entry_2 (Param : in out Param_Type) ;   -- Entry point with parameters
end Task_Name;

task body Task_Name is
begin
   accept Entry_1 (Param : in Param_Type);

   -- Statement(s) executed after rendezvous with Entry_1

   loop
      select
        accept Entry_2 (Param : in out Param_Type)
        do
           -- Statement(s) executed during the rendezvous
        end Entry_2;

         -- Statement(s) executed after Entry_2 rendezvous complete
      or
         delay 1.0 ;
         -- Statement(s) executed if no rendezvous within 1 second
      end select;
   end loop;

end Task_Name;
```

Exceptions

The Predefined Exceptions

Constraint_Error	-	Raised when something goes out of range
Numeric_Error	-	Raised when an arithmetic operation fails (Ada83 only)
Storage_Error	-	Raised if we run out of memory
Tasking_Error	-	Raised when the tasking rules are broken
Program_Error	-	Raised if the control structure of the language is violated

Declaring and Handling Exceptions

```
My_Exception_1 : exception ;
My_Exception_2 : exception ;

begin

  -- Statement(s)

exception

  when Constraint_Error | Numeric_Error  => Deal_With_Constraint_Error ;
  when My_Excpetion                      => Deal_With_My_Error ;
  when others                            =>  Deal_With_Other_Exceptions;
                                               raise;  -- Raise exception to next block

end;
```

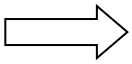
Representation

```
type Byte_Type is range 0..2**8-1 ;
for Byte_Type'Size use 8;  -- Objects of byte type only occupy 8 bits


type Mode_Type is (Off , Standby, Test, On);
for Mode_Type'Size use 4;
for Mode_Type use (Off      => 2#0001# ,   -- Represented as 1
                   Standby  => 2#0010# ,   -- Represented as 2
                   Test     => 2#0100# ,   -- Represented as 4
                   On       => 2#1000# );  -- Represented as 8

type Array_Type is array (1..8) of Boolean;
pragma Pack (Array_Type);
for Array_Type'Size use 8;  -- Objects of array type only occupy 8 bits
```

```
type Record_Type is
record
  Mode   : Mode_Type;
  Time   : Time_Type;
  Height : Height_Type;
end record;
```



```
for Record_Type use
record
  Mode   at 0 range 0 .. 3  ;  -- Byte 0 bits 0..3
  Time   at 0 range 4 .. 8  ;  -- Byte 0 bits 4..8
  Height at 0 range 9 .. 15;   -- Byte 0 bits 9..15
end record;
```

Attributes

Discrete Types

```
type Vehicle_Type is (Car, Truck, Motorbike) ;
type Speed_Type is range 0..120;

Vehicle_Type'First;          -- Car
Speed_Type'Last;             -- 120
Vehicle_Type'Succ (Car);     -- Truck
Speed_Type'Pred (70);        -- 69
Vehicle_Type'Pos (Truck);    -- 1
Vehicle_Type'Val (2);        -- Motorbike
```

Array Types

```
type Key_Type is range 0..19;
Active_Keys : array (Key_Type) of Boolean ;
……..
Active_Keys'First      -- Result = 0
Active_Keys'Last       -- Result = 19
Active_Keys'Length     -- Result = 20
Active_Keys'Range      -- Result = 0 .. 19  (can be used to iterate through an array)
```